

Digital Design

Moore FSM Vs Mealy FSM

A. Sahu

Dept of Comp. Sc. & Engg.

Indian Institute of Technology Guwahati

Comparison

Mealy FSM Vs Moore FSM

Recap: FSM Types

- Two main types of FSMs
 - Moore FSM : output is only function of state
 - Mealy FSM: output is function of state and inputs

Recap: Set Theoretic Description

Moore Machine is an ordered quintuple

$$\text{Moore} = (\mathbf{S}, \mathbf{I}, \mathbf{O}, \delta, \lambda)$$

where

$$\mathbf{S} = \{s_1, s_2, \dots, s_n\}$$

$$\mathbf{I} = \{i_1, i_2, \dots, i_m\}$$

$$\mathbf{O} = \{o_1, o_2, \dots, o_l\}$$

$$\delta = \mathbf{S} \times \mathbf{I} \rightarrow \mathbf{S}$$

$$\lambda = \mathbf{S} \rightarrow \mathbf{O}$$

$\mathbf{S}$ = finite set of state ($\neq \emptyset$)

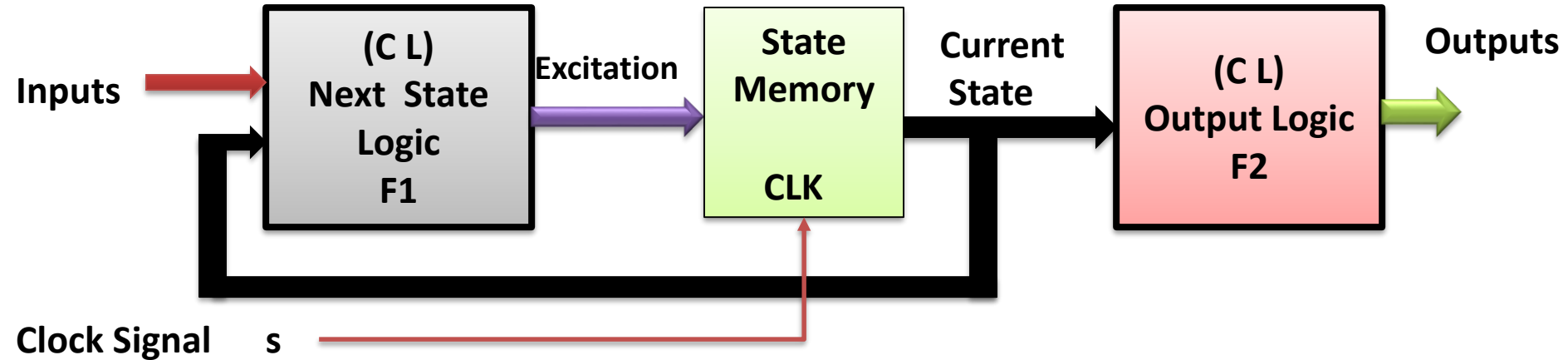
$\mathbf{I}$ = finite set of inputs ($\neq \emptyset$)

$\mathbf{O}$ = finite set of output ($\neq \emptyset$)

δ = next state function

λ = output function

Recap: Moore Machine: Moore FSM



Next state = $F_1(\text{current state, inputs})$
Output = $F_2(\text{current state})$

Recap: Set Theoretic Description

Mealy Machine is an ordered quintuple

$$\text{Mealy} = (\mathbf{S}, \mathbf{I}, \mathbf{O}, \delta, \beta)$$

where

$$\mathbf{S} = \{ s_1, s_2, \dots, s_n \}$$

$$\mathbf{I} = \{ i_1, i_2, \dots, i_m \}$$

$$\mathbf{O} = \{ o_1, o_2, \dots, o_l \}$$

$$\delta = \mathbf{S} \times \mathbf{I} \rightarrow \mathbf{S}$$

$$\beta = \mathbf{S} \times \mathbf{I} \rightarrow \mathbf{O}$$

$\mathbf{S}$ = finite set of state ($\neq \emptyset$)

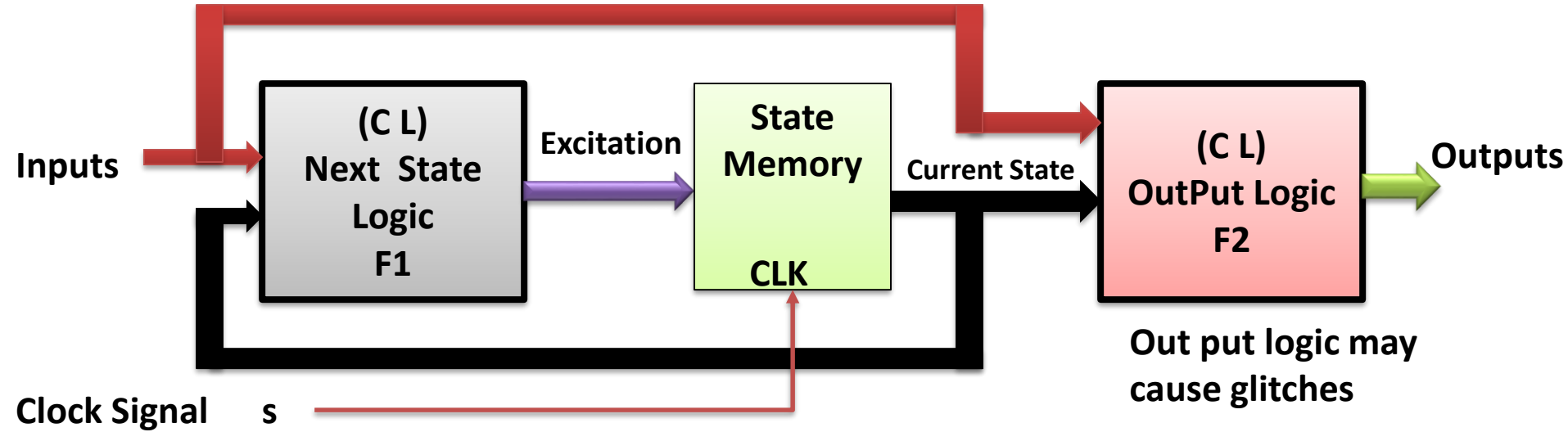
$\mathbf{I}$ = finite set of inputs ($\neq \emptyset$)

$\mathbf{O}$ = finite set of output ($\neq \emptyset$)

δ = next state function

λ = output function

Recap: Mealy Machine: Mealy FSM

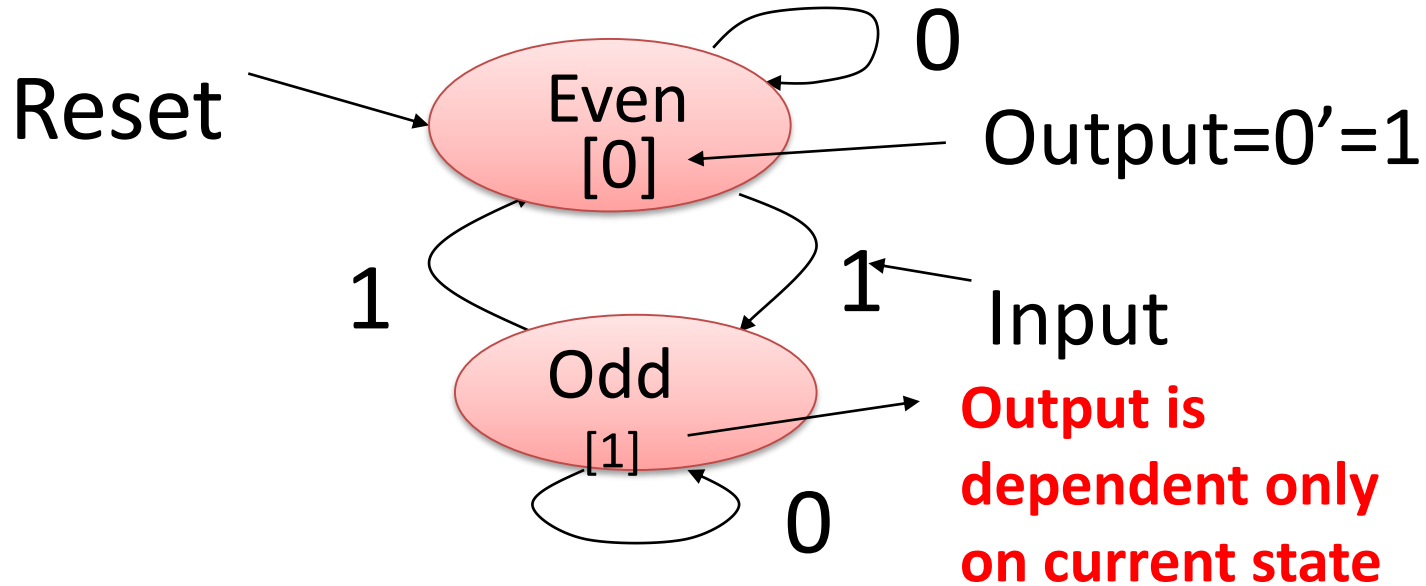


State Storage= Set of n FFs
 2^n State can be stored

Next state = $F_1(\text{current state, inputs})$
Output = $F_2(\text{current state, inputs})$

Moore FSM : Parity Encoder

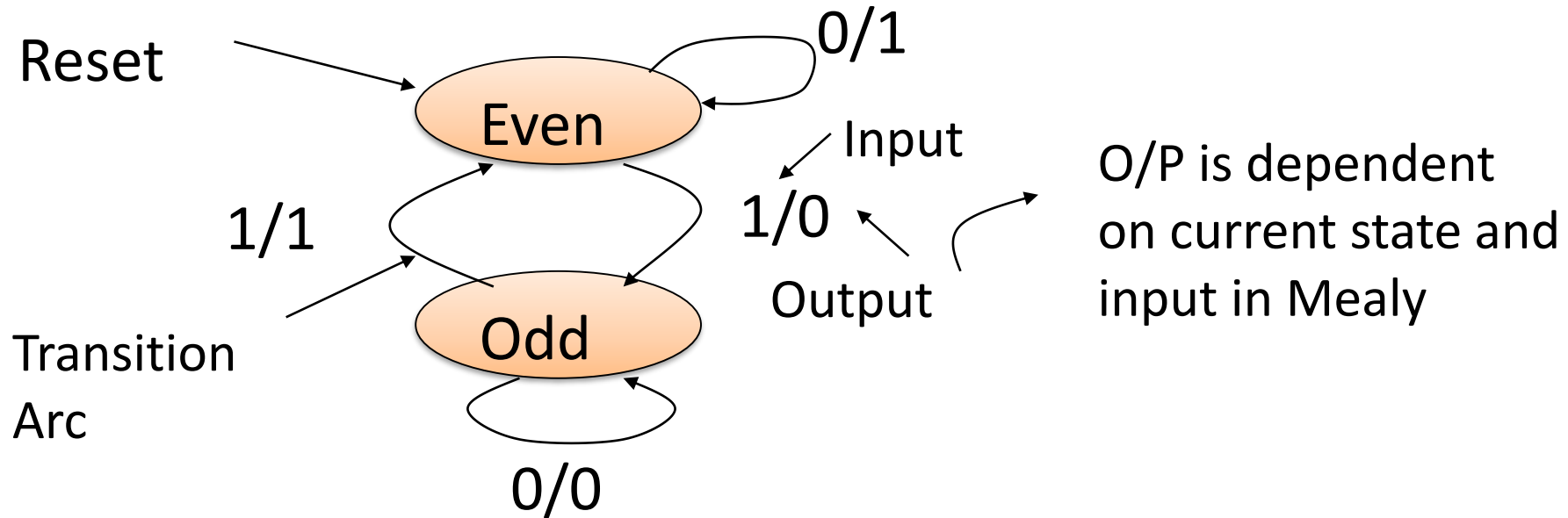
- Input: 1 or 0 // entering as stream
- Output: output a 1 when total number of 1 is even



Output is associated with the state and hence appears after the state transition take place.

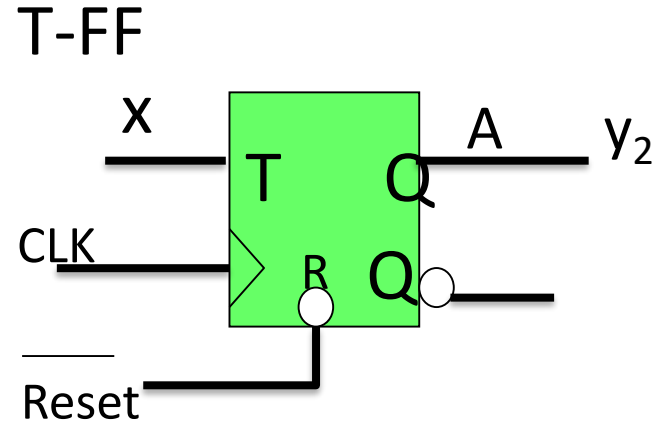
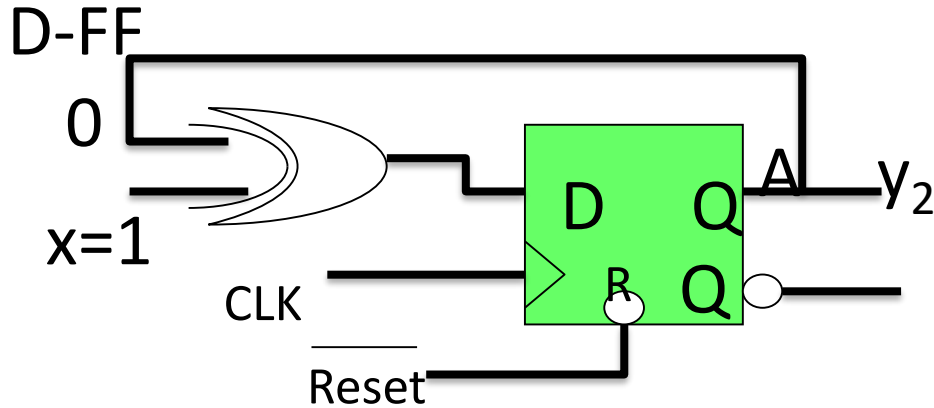
Example of Moore & Mealy Machine

- Parity Checker : Mealy FSM



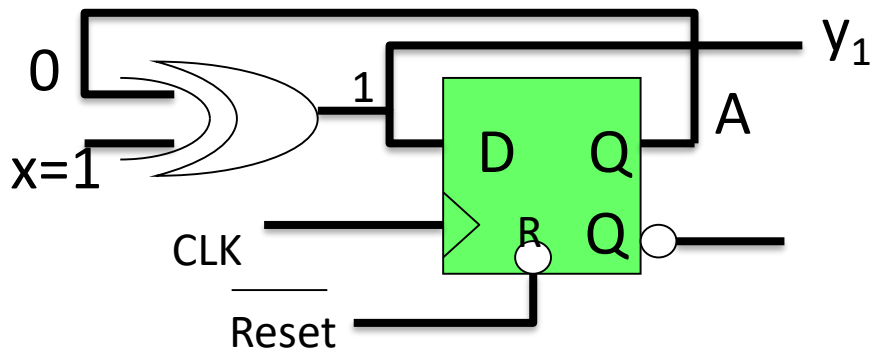
Mealy Machine: Output is associated with the state transition, and appears **before** the state transition is completed (by the next clock pulse).

Parity Checker: Moore M/C Implementation

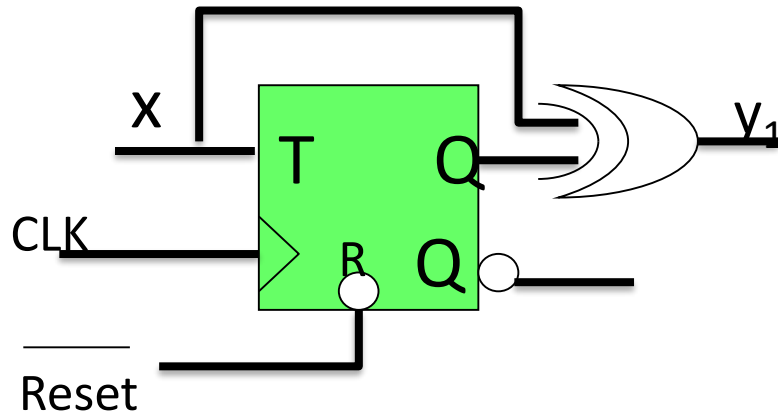


Moore O/P is synchronized with clock

Parity Checker: Mealy M/C Implementation



D-FF



T-FF

Mealy O/P is not synchronized with clock.

Comparison : Mealy FSM Vs Moore FSM

- **Mealy** machines have **less states**
 - Outputs are on transitions ($S \times I$) rather than states (n)
- **Moore** machines are **safer** to use
 - Outputs change at clock edge (always one cycle later)
- Mealy machines: input change can cause output change as soon as logic is done
 - A **big problem** when two machines are interconnected
 - Asynchronous feedback **may** occur if one isn't careful

Comparison : Mealy FSM Vs Moore FSM

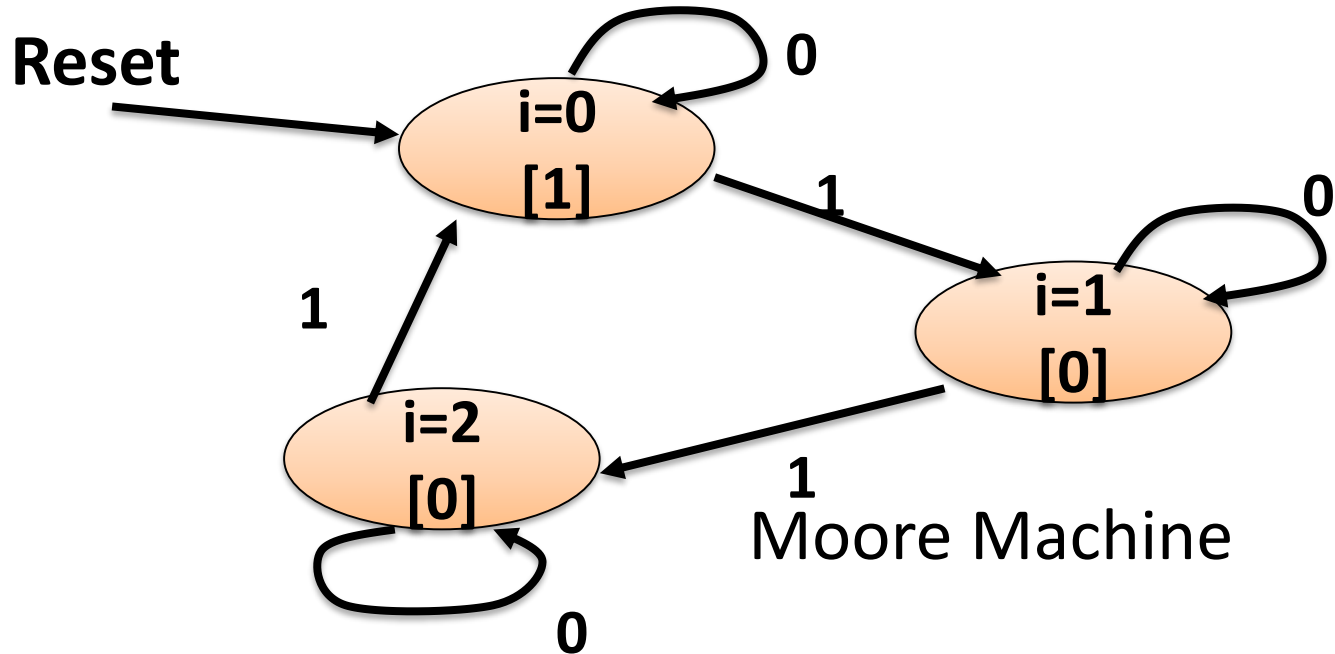
- **Mealy** machines **react faster** to inputs
 - react in same cycle – don't need to wait for clock
 - outputs **may** be considerably shorter than the clock cycle
 - in Moore machines, more logic **may** be necessary to decode state into outputs – there **may** be more gate delays after clock edge

FSM Example : Mealy FSM vs Moore FSM

- Design a system that
 - Output $X = '1'$ whenever it receives a multiple of 3 numbers of 1's on a serial input line x .
 - i.e. 0, 3, 6, 9, etc. number of 1's

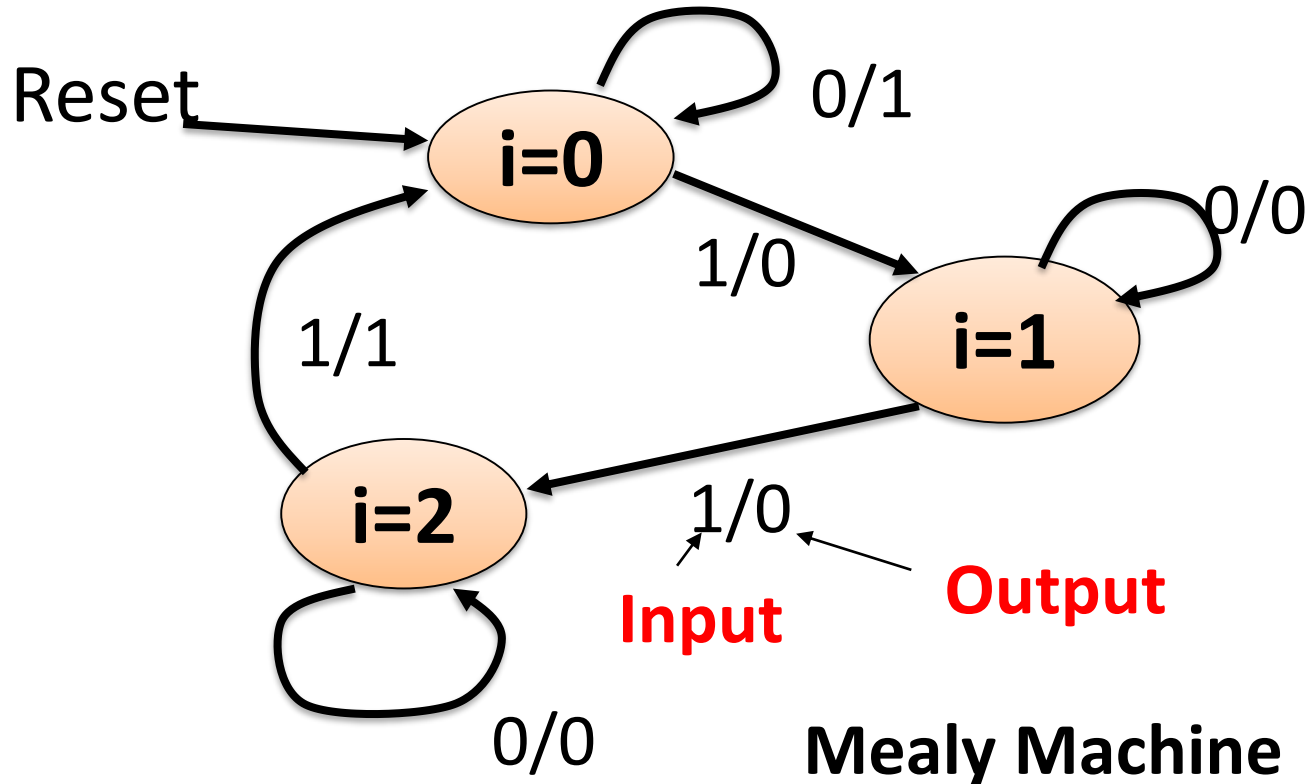
FSM Example: Moore Machine

- Design a system that outputs a '1' whenever it receives a multiple of 3 # of 1's (i.e., 0, 3, 6, 9, etc. # of 1's) on a serial input line x.



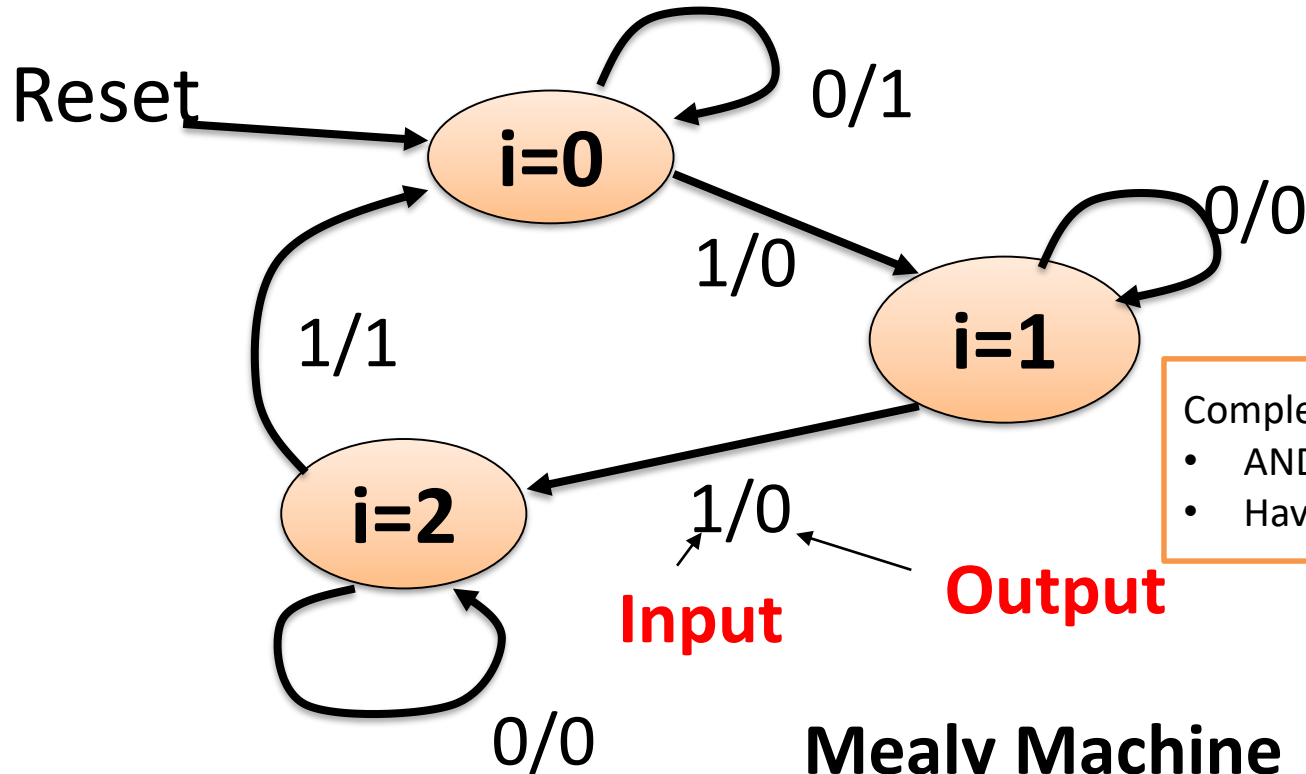
FSM Example: Mealy Machine

- Design a system that outputs a '1' whenever it receives a multiple of 3 # of 1's (i.e., 0, 3, 6, 9, etc. # of 1's) on a serial input line x.



FSM Example: Mealy Machine

- Design a system that outputs a '1' whenever it receives a multiple of 3 # of 1's (i.e., 0, 3, 6, 9, etc. # of 1's) on a serial input line x.



Complete and correct

- AND and OR Properties
- Have all edges: S X I

Mealy Machine

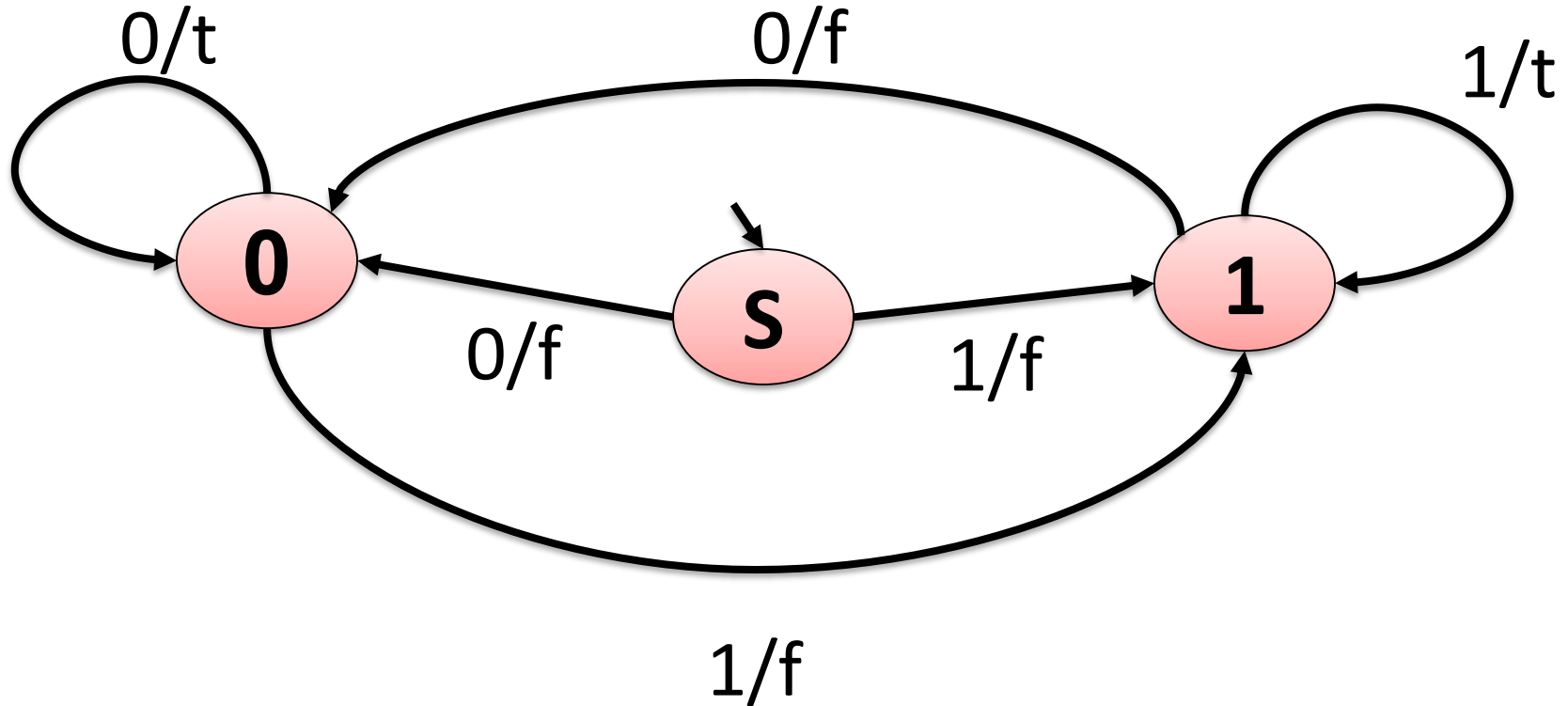
Mealy Vs Moore : Problem

Design a system that outputs **true** : at least two last input are same

Solutions to Problem

- Mealy implementation
 - Needs only 3 states
- Moore implementation
 - Needs 2 more almost duplicated states just in order to the output value t there.

FSM: Mealy Machine

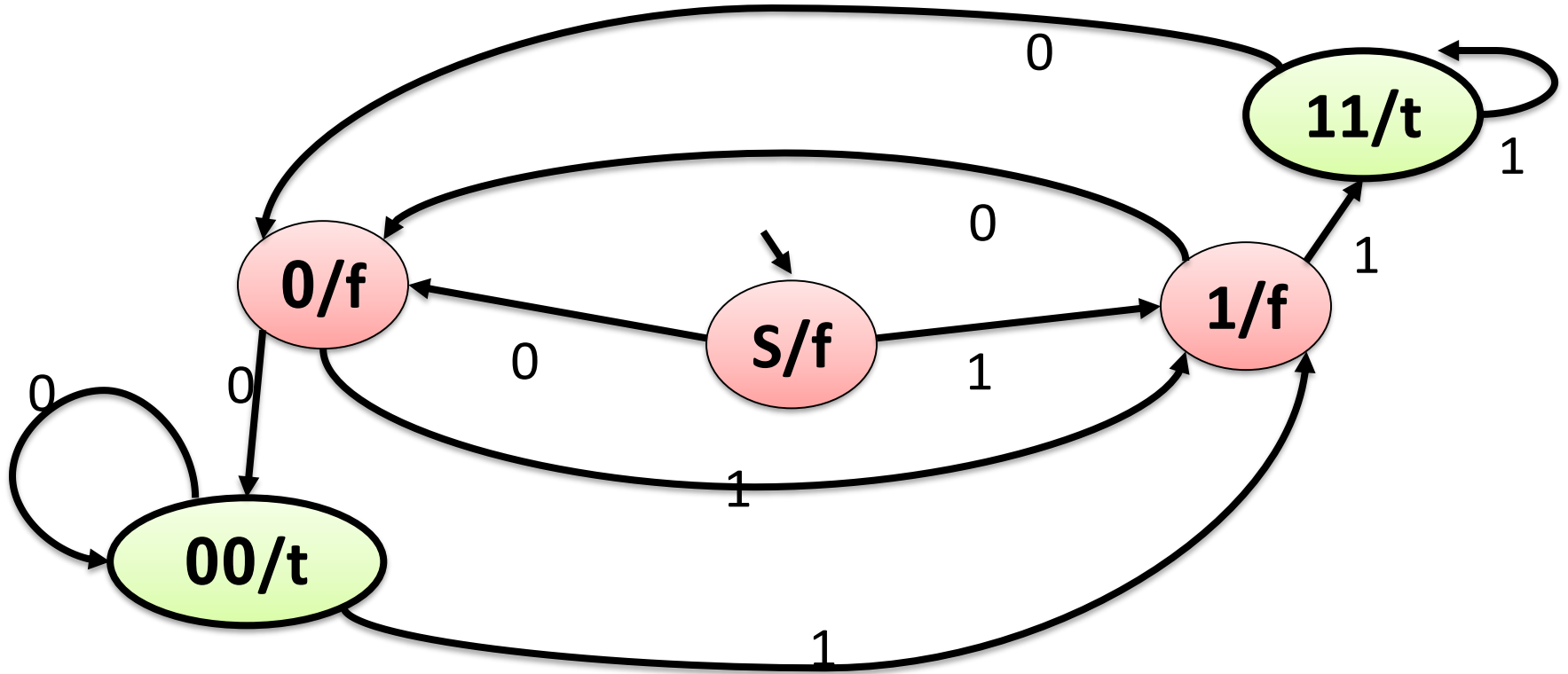


FSM: Moore Machine

Outputs **true** if at least 2 last input digits are same

Is it possible to design Moore
FSM of the same problem with 3
state ?

FSM: Moore Machine



Thanks